

《网络与通信》课程实验报告

实验 2: Socket 通信编程

姓名	赵文字	院系	计算机学院	学号	23121769
任课教师	张瑞	指导教师	张瑞		
实验地点	计算机大楼 706	实验时间	2025/11/5		
实验课表现	出勤、表现得分(10)		实验报告 得分(40)	实验总分	
	操作结果得分(50)				

实验目的:

1. 掌握 Socket 编程过程;

2. 编写简单的网络应用程序。

实验内容:

利用你选择的任何一个编程语言, 分别基于 TCP 和 UDP 编写一个简单的 Client/Server 网络应用程序。具体程序要求参见《实验指导书》。

要求以附件形式给出:

● 系统概述: 运行环境、编译、使用方法、实现环境、程序文件列表等;

● 主要数据结构;

● 主要算法描述;

● 用户使用手册;

● 程序源代码;

实验要求: (学生对预习要求的回答) (10 分)	得分:
● Socket编程客户端的主要步骤 1、创建套接字: 同下方的服务器端, 选择地址族和协议类型。 2、设置选项: 如超时、非阻塞、TCP_NODELAY 等。 3、连接: 向目标服务器的 IP:端口发起连接(阻塞或非阻塞)。 4、读写数据: 发送请求, 接收响应。注意同样要处理消息边界、部分读写。 5、处理并发/异步: 客户端若需同时与多台服务器通讯可用多线程、异步IO或事件循环。 6、重连与错误处理: 处理连接失败、超时、短连接 vs 长连接的选择与重连策略(指数退避等)。 7、关闭 socket: 完成后关闭连接并释放资源。 8、安全: 若需要加密通信, 使用 TLS/SSL, 并校验证书与主机名。 9、性能调优: 调整发送/接收缓冲、Nagle 算法、并发连接数等。 10、进行测试: 可以用 telnet、nc/netcat、curl (对HTTP) 调试连接与数据交互。 ● Socket编程服务器端的主要步骤 1、创建套接字: 指定地址族(如 IPv4 AF_INET / IPv6 AF_INET6)和协议类型(TCP 用 SOCK_STREAM, UDP 用 SOCK_DGRAM)。 2、设置选项: 例如 SO_REUSEADDR 以便快速重启; 在需要时设置非阻塞模式或超时。 3、绑定: 把 socket 绑定到本地 IP 和端口(例如 0.0.0.0:8080 或具体网卡地址)。 4、监听: 对于 TCP, 把 socket 转为监听状态并设置等待队列长度。 5、接受连接: 阻塞或非阻塞地接受客户端连接, accept 返回一个用于通信的新 socket	

(以及客户端地址)。

6、读写数据：在已连接的 `socket` 上接收请求并发送响应。

7、针对长连接处理心跳、超时。

8、并发处理：常见方式为多线程/多进程、线程池、事件驱动或异步框架（如 `asyncio`、`Netty`）。

9、关闭连接/清理资源：在合适时机关闭通信 `socket`，并在服务停止时关闭监听 `socket`。

10、异常与日志：捕获网络错误（连接中断、超时）、记录日志，并做重试或资源回收。

实验过程中遇到的问题如何解决的？（10 分）

得分：

问题 1：TCP 客户端与 TCP 服务端不在同一个局域网下，无法调通程序。

解决方案：客户端与服务端连接到同一个手机热点下，通过 `ipconfig` 命令查看 `ip` 地址，并通过 `ping` 命令测试是否连通。

客户端 `ipconfig`:

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . : shu.edu.cn
IPv6 地址 . . . . . : 2408:840d:7b00:74b9:5efd:92de:d37e:8361
临时 IPv6 地址 . . . . . : 2408:840d:7b00:74b9:9df8:b99d:f6b1:bb32
本地链接 IPv6 地址 . . . . . : fe80::fc8e:dacc:72c:7bc7%10
IPv4 地址 . . . . . : 172.20.10.4
子网掩码 . . . . . : 255.255.255.240
默认网关 . . . . . : fe80::2cc2:53ff:fea4:4764%10
                        172.20.10.1
```

服务端 `ipconfig`:

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :
IPv6 地址 . . . . . : 2408:840d:7b00:74b9:78ee:d862:5cbb:1bd1
临时 IPv6 地址 . . . . . : 2408:840d:7b00:74b9:fc29:f28e:24f0:1e8a
本地链接 IPv6 地址 . . . . . : fe80::d12d:f87f:bbb4:17f9%22
IPv4 地址 . . . . . : 172.20.10.2
子网掩码 . . . . . : 255.255.255.240
默认网关 . . . . . : fe80::8205:88ff:fe3f:e11e%22
                        fe80::2cc2:53ff:fea4:4764%22
                        172.20.10.1
```

客户端 `ping`:

C:\Users\fdfx_>ping 172.20.10.2

```
正在 Ping 172.20.10.2 具有 32 字节的数据:
来自 172.20.10.2 的回复: 字节=32 时间=6ms TTL=128
来自 172.20.10.2 的回复: 字节=32 时间=4ms TTL=128
来自 172.20.10.2 的回复: 字节=32 时间=5ms TTL=128
来自 172.20.10.2 的回复: 字节=32 时间=4ms TTL=128
```

```
172.20.10.2 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 4ms, 最长 = 6ms, 平均 = 4ms
```

服务端 `ping`:

```
正在 Ping 172.20.10.4 具有 32 字节的数据 :
来自 172.20.10.4 的回复: 字节=32 时间=5ms TTL=128
来自 172.20.10.4 的回复: 字节=32 时间=4ms TTL=128
来自 172.20.10.4 的回复: 字节=32 时间=5ms TTL=128
来自 172.20.10.4 的回复: 字节=32 时间=4ms TTL=128

172.20.10.4 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 4ms, 最长 = 5ms, 平均 = 4ms
```

问题 2: TCP 服务端预设了一部分的用户名与密码, TCP 客户端输入的用户名密码与服务端中预设的部分不符时, 出现报错: “登陆失败, 无法连接”。

```
Connected to server
Please enter username:user1
Please enter password:passsss
Login failed. Disconnecting...
```

进程已结束, 退出代码为 0

解决方案: 后来发现这样的登陆失败是正常的, 客户端输入的用户名与密码必须要与服务端中的某个匹配。有两种解决办法: 第一种是输入服务端现有的用户名与密码; 第二种是在服务端增加该用户名与密码, 然后再次在客户端尝试连接即可。

问题 3: 服务端与客户端的端口不一致时, 无法连通。

解决方案: 将服务端与客户端的端口配置为同一端口, 同时该端口空闲、不被其他进程调用即可。

问题 4: 服务器接收数据超时导致客户端无响应, 一段时间后, 弹出进程错误。

导致该问题可能有两个原因, 一个是服务器繁忙, 没有及时的返回数据; 二是服务器有返回数据, 但是由于网络繁忙, 客户端没能及时接收到数据。开始, 程序在等待数据接收, 从用户的视觉则是卡机状态, 当长时间没有响应后, 在系统保护机制下, 系统会自动发送一个 signal 信号杀掉该进程, 此时会弹出一个询问是否关闭程序的进程错误对话框。

解决方案: 可以通过在程序中自设超时时间, 避免延时过长造成卡机的假现象, 同时逃避系统的查杀。使用异常处理方法, Catch 到超时错误, 并通知用户访问超时。

本次实验的体会 (结论) (10 分)

得分:

本次计网的实验围绕 Socket 通信编程展开, 在学习了网络层后, 做了一次这样的实验是一次将抽象的网络协议理论转化为具体应用能力的宝贵实践。我与另外一个同学合作通过分别基于 TCP 和 UDP 协议编写 Client/Server 应用程序, 不仅巩固了课堂上学到的知识, 同时也对网络编程的细节、挑战及其现实意义有了深刻而全面的理解。

首先来说, 实验过程让我真切体会到理论与实践的结合点与差异。在理论学习中, TCP 的三次握手、可靠传输, UDP 的无连接、高效性等概念是清晰而明确的, 不需要过多思考, 只需要将这些概念与理论牢记在心即可。然而, 在编码实现阶段, 这些特性转化为了具体的编程步骤和需要处理的异常情况。例如, TCP 编程中, 我深刻理解了“面向连接”意味着必须先建立稳定的通信信道, 其编程模型是严谨的“请求-响应”流程; 而 UDP 编程则体现

了“无连接”的灵活性，数据包即发即走，但同时也带来了数据可能丢失、乱序的可靠性挑战，需要我在应用层设计简单的确认机制。这种从协议特性到代码逻辑的映射，极大地深化了我的认知。 其次，本次实验极大地锻炼了我分析问题和解决问题的能力。实验报告中记录的几个关键问题，正是网络编程中常见的“陷阱”。第一，网络连通性问题。最初因客户端与服务端不在同一局域网而无法通信的经历，让我认识到 IP 地址配置、子网划分和网关路由在实际网络环境中的重要性。通过使用 <code>ipconfig</code> 和 <code>ping</code> 命令进行诊断，我学会了基本的网络故障排查方法，这是成为一名合格网络工程师的基础技能。第二，应用逻辑错误。在实现简单的用户名密码认证时，由于客户端输入与服务端预设不匹配导致的“登录失败”，让我意识到服务器端业务逻辑的严谨性以及客户端与服务端之间数据格式约定（如 <code>username:password:message</code>）的必要性。第三，资源与配置管理。端口号不一致导致连接失败的问题，警示我网络编程中“地址+端口”的唯一性标识至关重要。而服务器响应超时引发的客户端假死问题，则引导我思考如何通过设置 <code>socket</code> 超时参数来提升用户体验和程序健壮性，这是编写高质量网络应用不可忽视的一环。 再者，通过亲手编写代码，我对 TCP 和 UDP 的核心差异有了更直观的对比。TCP 如打电话，需要先拨号接通（建立连接），保证双方每一句话都清晰传达（可靠有序），适合如文件传输、网页浏览等对准确性要求高的场景。UDP 如寄明信片，写好地址直接投递（无连接），高效快速但可能丢失（不可靠），非常适合视频直播、在线游戏等实时性要求高于可靠性的场景。在代码层面，TCP 使用 <code>ServerSocket</code> 和 <code>Socket</code> 类，通过流进行数据读写；UDP 则使用 <code>DatagramSocket</code> 和 <code>DatagramPacket</code>，以数据报为单位进行发送和接收。这种差异直接决定了它们在不同应用场景下的选择。 最后，这次实验也是一次完整的软件开发流程体验。从需求分析、环境搭建、编码实现、调试测试到撰写报告，我系统地实践了一个小型网络应用的诞生过程。它让我认识到，一个成功的程序不仅在于功能的实现，更在于对异常情况的周全处理、对用户交互的友好设计以及对网络环境的良好适应性。 总而言之，本次 <code>Socket</code> 通信实验是一次富有成效的学习旅程。它让我跳出了纯理论的框架，在动手实践中将知识内化为技能。我所获得的不仅是 Java 中 <code>DatagramSocket</code>、<code>InetAddress</code> 等类的使用技巧，更重要的是建立了一套解决网络通信问题的思维方法。这为我今后学习更复杂的网络技术（如 HTTP、RPC 框架等）打下了坚实的基础，也让我对互联网世界的运作方式充满了更浓厚的兴趣与探索欲。	
思考题：（10 分）	
思考题 1：（4 分）	得分：
你所用的编程语言在 <code>Socket</code> 通信中用到的主要类及其主要作用。 答：我使用 Java 进行编程，编写了一个 UDP 客户端，主要使用了以下四个与 <code>Socket</code> 通信相关的类： 1: <code>DatagramSocket</code>，其作用为创建 UDP 套接字，用于发送和接收数据报包。在程序中，具体来说，我是这样使用的(<code>DatagramSocket socket = new DatagramSocket()</code>)，成功创建了客户端套接字。 2: <code>DatagramPacket</code>，其作用为封装数据报包，包含要发送/接收的数据、目标/源地址和端口。在程序中，具体来说，我使用了两次，一次是在发送包的时候（<code>DatagramPacket sendPacket</code>）封装要发送的数据,；另一次是在接受包的时候（<code>DatagramPacket receivePacket</code>）接收服务器响应。 3: <code>InetAddress</code>，其作用为表示 IP 地址，用于网络地址解析和操作。在程序中，具体来说，我是这样使用的（<code>InetAddress.getByIp(SERVER_IP)</code>）将字符串 IP 转换为 <code>InetAddress</code>	

对象

4: **Scanner**，其作用为读取用户输入（虽然这个类不是 **Socket** 专用类，但在网络通信中用于交互，所以还是把它放在我的回答中了）在我的程序中，就主要用于读取用户名、密码和消息内容。

综合来说，我的程序执行的流程为：创建 **DatagramSocket** 建立 UDP 连接、通过 **InetAddress** 解析服务器地址、使用 **Scanner** 获取用户认证信息和消息、用 **DatagramPacket** 封装数据并发送、用另一个 **DatagramPacket** 接收服务器响应、处理并显示响应结果。

```

1  import java.io.*;
2  import java.net.*;
3  import java.util.Scanner;
4
5  public class UDPClient {
6      public static void main(String[] args) {
7          final String SERVER_IP = "172.20.10.2"; // 服务器IP
8          final int PORT = 6789;
9
10         try (DatagramSocket socket = new DatagramSocket();
11             Scanner scanner = new Scanner(System.in)) {
12
13             InetAddress serverAddress = InetAddress.getByName(SERVER_IP);
14             System.out.println("Connected to server at " + SERVER_IP + ":" + PORT);
15
16             // 身份验证
17             System.out.print("Enter username: ");
18             String username = scanner.nextLine();
19             System.out.print("Enter password: ");
20             String password = scanner.nextLine();
21
22             System.out.print("Enter your message: ");
23             String message = scanner.nextLine();
24
25             String fullMessage = username + ":" + password + ":" + message;
26             byte[] sendData = fullMessage.getBytes();
27             DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, PORT);
28             socket.send(sendPacket);

```

```

5  public class UDPClient {
6      public static void main(String[] args) {
7
8          // 发送数据
9          String fullMessage = username + ":" + password + ":" + message;
10         byte[] sendData = fullMessage.getBytes();
11         DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, PORT);
12         socket.send(sendPacket);
13
14         // 接收响应
15         byte[] receiveData = new byte[1024];
16         DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
17         socket.receive(receivePacket);
18
19         String response = new String(receivePacket.getData(), 0, receivePacket.getLength());
20         System.out.println("Server response: " + response);
21
22         System.out.println("Press Enter to exit...");
23         scanner.nextLine();
24     } catch (IOException e) {
25         System.out.println("Client exception: " + e.getMessage());
26     }
27 }

```

思考题 2: (6 分)

得分:

说明 TCP 和 UDP 编程的主要差异和特点。

TCP 是一种面向连接、可靠的、基于字节流的传输协议。在编程中，TCP 通信需要经过“创建套接字（socket）—绑定（bind）—监听（listen）—接受连接（accept）—收发数据（send/recv）—关闭连接（close）”等步骤。通信前必须建立连接（即三次握手），通信结束时还需要释放连接（即四次挥手）。TCP 在传输过程中提供了错误检测、重传、确认应答、流量控制和拥塞控制等机制，能够保证数据的可靠、有序传输。但由于这些机制的存在，TCP 的传输效率和实时性相对较低，系统开销较大。

UDP 是一种无连接、不可靠的、基于数据报的传输协议。在编程中，UDP 通信不需要建立和释放连接，主要通过“创建套接字（socket）—绑定（bind）—发送与接收数据报（sendto/recvfrom）—关闭套接字（close）”等步骤完成。UDP 不提供确认和重传机制，发送端发送一个数据报，接收端一次性接收一个完整数据报，因此不存在粘包或拆包问题。UDP 的开销小、速度快，适用于对实时性要求高、允许少量数据丢失的应用场景，如视频会议、网络游戏、语音通话等。

综合来看，TCP 适用于对数据传输可靠性要求高的场景（如文件传输、网页访问、电子邮件等），而 UDP 适用于对实时性要求高、能容忍部分数据丢失的场景。两者在编程实现中最大的区别在于：TCP 是面向连接、基于字节流的可靠传输，UDP 是无连接、基于数据报的不可靠传输。

指导教师评语：

日期：